

Pandoogle: a Tiny Pandas Helper

DSC 360 F25 Lab 04

A.J. Daniels and Madeleine Arganbright

Overview

We began by embedding each chunk of our corpus (the pandocs, where each doc was its own “chunk”). Then by comparing the chunk embeddings to the prompt embedding, we returned the top5 passages with the highest cosine similarities, making sure to exclude any chunk which had a similarity index under 0.45. We had our generation model generate a response using the top5 docs and the user’s query, asking it to also cite its source by including the corresponding dataframe symbol. Models used are listed in methods.

Problem

The purpose of this tool was to use a specialized corpus made from the pandas help docs to return specific information for panda related-queries. We introduced a safeguard in the form of a similarity cutoff, to ensure that queries which did not yield results with satisfactory cosine similarity scores just returned a message saying it couldn’t find the answer, rather than hallucinating an inappropriate response.

Methods

Embedding model used: qwen3-embedding:8b

Generation model used: gemma3:4b

Prompt: The user has asked you a question about using pandas. Here is a string containing the most relevant panda docs with a symbol, signature, description, and a similarity score (higher number is a closer answer). If the context does not contain relevant information, print "Couldn't find an answer, please revise your query and try again". Use this context string to answer the question as accurately as possible: {context} Also make sure to include the dataframe symbol that corresponds to the document you are using to answer the question. Here is the question: {userInput}.

Text embedded for each symbol: "symbol: " + line['symbol'] + '\n' + "signature: " + line['signature'] + '\n' + "doc: " + line['doc'] + '\n'

Enforcement of similarity cutoff: We included an if statement to catch cases in which the top5 list was empty (meaning none of the top hits were above the similarity threshold), in which case we returned "Couldn't find an answer, please revise your query and try again." We also included a line in our prompt which also had the model return this sentence if it felt the docs provided did not include enough information, as another safeguard against hallucinations.

Experiments

After running tests with nonsense queries, we decided to increase the similarity cutoff to 0.45, since completely nonsense inputs (e.g. “bla bla bla”) still returned similarities around the 41% range. We also experimented with different ways to ask the model to “cite” its sources in a way that was user-legible. (The model initially responded with json objects when just asked to also respond with which doc the information was coming from, so we revised our prompt to have it specifically include the

dataframe symbol, in the hopes that it would return just “dataFrame.add” or “add” instead of “{ symbol: ‘DataFrame.add’, signature: ‘(self, other...’” We still ended up only getting citations around half of the time, but they were always referencing pandadocs which actually existed. Our prompts are below, which all come from the suggestions on moodle.

NOTE: We did include an additional prompt which included “how to” before the question, to see if there was a difference in accuracy when the query was formulated as a question. We did not see a significant difference in the quality of response generated, but the time more than doubled as a result (as compared to our “control”, which was the same prompt without “how to” at the start).

Results

Prompt	Expected response	Hit@3 (was it in the context/response)	Did the code run?	Time	Cites correct doc?	Score *
“Sort by two columns (one ascending, one descending).”	sort_values	Yes	Yes	76.4108 seconds	Yes	10/10
“Left-join two frames on id.”	pd.merge(df1, df2, ...)	Yes	Yes	97.1620 seconds	No - didn’t cite anything	8/10
“Read CSV from a URL with ; delimiter.”	df = pd.read_csv(url, sep=';')	Yes	Yes	132.0894 seconds	No - cites testing_with_csv_dialect instead of read_csv	7/10
“Pivot wider (categories → columns), summing values.”	pivot_table	Yes	Yes	94.6084 seconds	No - didn’t cite anything	8/10
“How to pivot wider (categories	pivot_table	Yes - it suggests using	Yes	250.9267 seconds	No- didn’t cite anything	7/10

→ columns), summing values.”		.pivot, but also mentions pivot_table as a possible alternative				
“Drop rows with any NA and show the first 10.”	dropna() and head()	Yes, mentions using both	No- did not include import for numpy (after import was added, code ran as expected)	68.1698 seconds	Cites dropna	5.5/10
“Convert a column to datetime with a custom format.”	pd.to_datetime() format	Yes	Yes	113.7521 seconds	Cites to_datetime(), and also writes that “it [presumably the model] has linked to the correct document in the chain” (as part of the response, it evaluated its own response)	10/10
“I get SettingWithCopyWarning when assigning—what should I use instead?”	.loc or .iloc	Yes, mentions .loc	NA - it did give some suggested commands, just not full example code	48.5504 seconds	No - didn't cite anything	5/7
“Filter rows where score > 90 and add a new column grade.”	np.where	No, top answer did not meet threshold	No code given	64.6757 seconds	NA	5/10 Didn't give expected response, but did correctly identify below similarity threshold that

						we set
--	--	--	--	--	--	--------

Safety check: the prompt “write me a poem about panda bears” returned : “Couldn't find an answer, please revise your query and try again”

* Score calculated by adding 4 points to the total if the correct command was returned, 3 points if the code ran, 1 point if the runtime was less than 2 minutes (no points added if it was over 2), and 2 points if it cited the correct doc. Total score was out of 10 for prompts which warranted example code, and out of 8 if prompts did not warrant example code (Like test 8).

Analysis

Our rubric consisted of the following categories, each intended to measure the usefulness and accuracy of the response: Hit@3 (points were received if the expected command appeared in the response, meaning it was included in the context in the generation prompt), quality of code (full points given if the code ran with no changes, no points if any changes needed to be made -for example a missing import statement), time taken to run (which was calculated using the time module, which kept track of the times the program started and ended and returned elapsed time after completion), and citations (points were given only if a correct citation was in the response). We did not identify any hallucinations, the model always responded with information from an existing panda doc, even when it didn't suggest the keyword we were looking for. We did not check that the information provided about each symbol was correct, but a cursory read-through did not yield any concerns.

Conclusion

Overall, our model did respond with the appropriate symbol for each query, and the generated code almost always worked as intended (with the exception of one case where the ollama import statement was missing, and one where example code was not included in the response).

The main limitation was the inconsistency in source citation- the model only returned with the doc the information came from around half of the time, and usually the citation was buried in the response, rather than being explicitly noted. In the future, we would like to revise our prompt to improve the model's citations, and have it respond with its sources more frequently. We would also like to experiment with different embedding and generation models to see which yield better results.

Acknowledgements and References

We had a lot of Google queries answered by the Google AI, which returned example code we used throughout our program (indicated with comments in the code). We also utilized the python docs, stackOverflow, kdnuggets.com, and Geeks for Geeks for syntax.

We also used Chatgpt to help debug when we were stuck, and it suggested a better way to format our prompt. The query and response are below, and the generated code is marked in our program :

Why is the model not recognizing the context string? it responds saying it's expecting it, but printing the string shows it's there.

```
# Generation!
context = docSearch()
print("context: ", context)
prompt = "The user has asked you a question about using pandas.
Here is a string containing the most relevant panda docs with a
symbol, signature, description, and"
" a similarity score (higher number is a closer answer).Use this
context string to answer the question as accurately as possible: " +
context + "Also return the symbol that corresponds to"
"the document you are using to answer the question. Here is the
question: " + userInput + "."
try:
    response = ollama.chat(model = GE_MODEL, messages = [{"role"
: "user", "content" : prompt}])
    r = response.message.content
    print(r)
except ollama.ResponseError as e:
    print("Error", e.error)
```



Nice question — I can see exactly the kind of bug you're running into.

It's a *Python string concatenation* issue caused by how your `prompt` string is written.

Let's break down what's happening 📌

Option 2 — Use triple quotes

python

Copy code

```
prompt = f"""
The user has asked you a question about using pandas.
Here is a string containing the most relevant panda docs with a symbol, signature, description
and a similarity score (higher number is a closer answer).
Use this context string to answer the question as accurately as possible:

{context}

Also return the symbol that corresponds to the document you are using to answer the question.
Here is the question: {userInput}.
"""
```